Ambient Adventures: Teaching ChatGPT on Developing Complex Stories

Zexin Chen*, Eric Zhou*, Kenneth Eaton, Xiangyu Peng, Mark Riedl

Georgia Institute of Technology, Atlanta, GA, 30332, USA

arXiv:2308.01734v1 [cs.CL] 3 Aug 2023

Abstract

Imaginative play is an area of creativity that could allow robots to engage with the world around them in a much more personified way. Imaginary play can be seen as taking real objects and locations and using them as imaginary objects and locations in virtual scenarios. We adopted the story generation capability of large language models (LLMs) to obtain the stories used for imaginary play with human-written prompts. Those generated stories will be simplified and mapped into action sequences that can guide the agent in imaginary play. To evaluate whether the agent can successfully finish the imaginary play, we also designed a text adventure game to simulate a house as the playground for the agent to interact.

1 Introduction

In recent years, the domain of agents has experienced extraordinary progress, driving the creation of intelligent machines that connect the realms of science fiction and reality. As researchers, engineers, and innovators collaborate, the evolution of agents keeps pushing the limits of technology. However, how do we ensure that agents have a persistent, yet non-intrusive presence in the household? Considering kids: they are never idle — they find ways to occupy their time through play and if that play is imaginative play, then the entire home becomes a playground. We propose to develop the computational capability for agents to engage in imaginative play and link that play to navigation through the home. This will increase the presence of the agent in the home without directly demanding attention from people, but also using curiosity to invite engagement.

Imaginative play is an exemplar of everyday human creativity in which real-world, mundane objects and locations act as substitutes for imaginary objects and locations as part of a pretend scenario(Zook, Magerko, and Riedl 2011). A terrarium can be a garden for growing magic seeds, a kitchen can be a laboratory, or a broom handle can be a light saber. Imaginative play is fundamental to human creativity. Computational systems that can engage in imaginative play can create a sense of presence and persona and provide opportunities for improvisational interactions.

*These authors contributed equally.

Copyright © 2023, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

In this paper, we are focusing on exploring how to guide an agent to execute imaginary play with large language models such as ChatGPT (OpenAI 2022). Text adventure games serve as useful test beds because they have also been demonstrated to transfer to visual and real-world domains (Wang et al. 2022; Shridhar et al. 2021; Peng, Riedl, and Ammanabrolu 2022).

2 Related Work

ext games are turn-based games where players read descriptions of the current scene for information and interact with short descriptions of actions (Côté et al. 2018; Wang et al. 2022; Shridhar et al. 2021). During the designing of the text adventure game, we follow the text game structure (Peng et al. 2023) to create a house consisting of rooms that contain objects as the realistic mappings of the imaginary play. The difference is that our adventure games don't have NPCs since we want to focus on whether the story of imaginary play can guide the agent instead of elevating the difficulty of interaction in the game.

ChatGPT is an LLM chatbot developed by OpenAI that can be interacted with in multiple ways, including giving prompts to write stories (OpenAI 2022). Many pre-trained LLMs have had success in story generation; by decomposing textual story data into a series of events, it has been found that these models can generate stories from these events that are more coherent and logical (Martin et al. 2018; Peng et al. 2022b,a). We aim to adopt this ability to train the model with prompting and generate stories that can guide the agent in imaginary play with ChatGPT.

3 Story Generation

A natural language story is generated automatically as an exemplar of the behavior the agent is to enact in imaginary play. Given the topic of the imaginary play, Large Language Models (LLMs) such as ChatGPT and GPT-4 are used to create the imaginary story, and the real-world objects are transformed into similar objects in an imaginary world — *imaginary objects*, to facilitate the imaginary play.

3.1 Imaginary Story Generation

Firstly, the agent scans the layout of each house to obtain the real-world objects and rooms, as well as their respective lo-

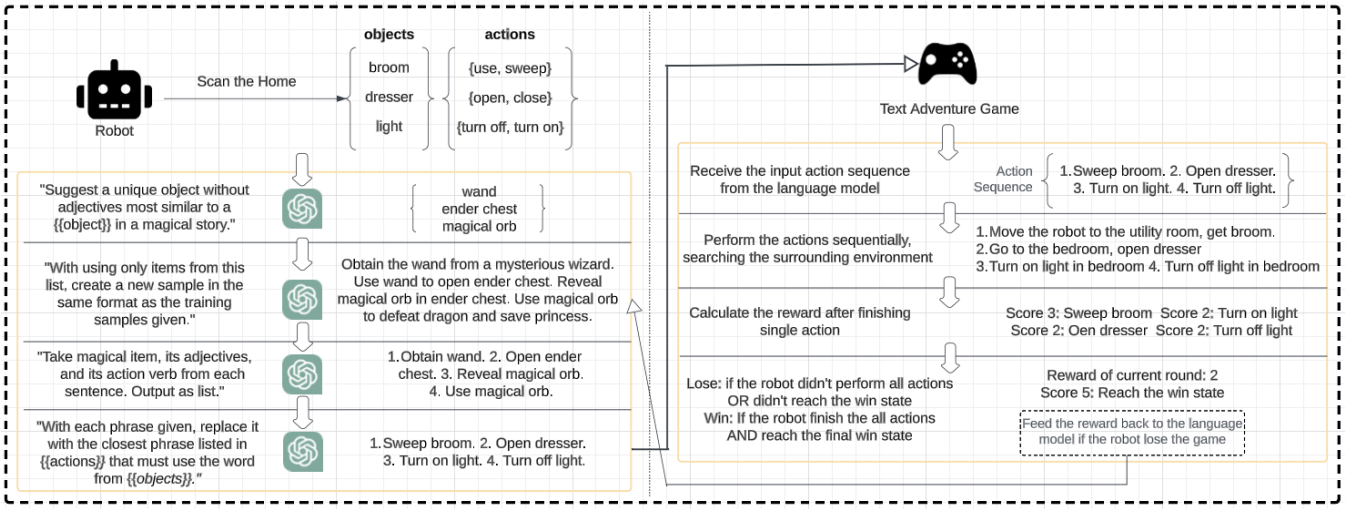

Figure 1: Pipeline Architecture for Text Game and ChatGPT. A sample iteration is demonstrated.

cations in the house. In this process, LLMs (ChatGPT) generate imaginary objects which match the setting of the imaginary world (whether that be magical, horror, etc.) that have similar characteristics to the object it was mapped from. For example, a "`broom`" can be transformed into a "`wand`" in the imaginary setting because they have similar shapes, are both made of wood and can be held. See Fig 1, where the first prompt by ChatGPT is used to map each original object to an imaginary one.

With each real-world object, we also obtain the admissible actions for each one – A_o – an admissible action refers to one that can be performed with that object. Let there be a set of real-world objects, such as ⟨"`broom`", "`dresser`", "`mug`"⟩. The set O_o denotes the set of real-world objects. For example, a "`broom`" in the house may have the set of admissible actions ⟨"`sweep`", "`pick up`"⟩. With this, LLMs are prompted to find the closest, most similar imaginary object for each item in O_o, based on the setting of the story. By this, we map all of our objects from the original O_o to the set of imaginary objects — O_n. Within any story, a topic, such as "saving a princess", is required for LLMs (ChatGPT) to aim to complete. A wide variety of topics were given based on the setting. For example, a magical setting could entail saving a princess, or a horror story would entail finding a key to escape.

To ensure that the generated story by ChatGPT can be simplified into phrases later, several training samples were given to it to ensure that it would be easy to do so. These training samples are short, 5-7 sentence imaginary stories that are concise and contain several random imaginary objects and a topic (such as "defeating the dragon"). Refer to Fig 1 and the second ChatGPT prompt which uses these training samples and a list of the imaginary objects to generate new samples to use. A common limitation of ChatGPT's story generation is that it will simply create a story where the agent immediately obtains every imaginary item in one sentence and follows by completing the topic. While this indeed works logically, it is far from interesting. With the training samples given, ChatGPT is prompted to at most add one new item in each sentence. For example, if ChatGPT is given 5 magical items and a topic to save a princess, it will initially get all 5 items in the first sentence and then save the princess in the second. With this restriction, ChatGPT has to find a logical way to use objects to get others and continue until it has enough objects to save the princess.

LLMs may sometimes be prompted to rewrite the story – instead of entirely rewriting the story ChatGPT will be prompted to continue it – refer to Table 1 for the second iteration. A sample iteration would involve the last sentence of the original imaginary story from Table 1 being removed and ChatGPT prompted to generate new sentences starting from this point to reach the intended topic. At this point, ChatGPT has successfully generated a story, and now this story needs to be distilled and translated back into admissible actions in the text game.

3.2 Mapping and Filtering

The reason for distilling the story is so that these actions can be given for the text game to easily understand. Each sentence in the imaginary story is distilled into a phrase by taking the one imaginary object in each sentence and the action verb that is associated with it. Refer to Figure 1 and the third ChatGPT prompt, as well as the *Simplified Story* in Table 1. If there is more than one object in a sentence, the newly obtained object is chosen. For example, if a sentence is "`open chest to reveal staff`", the distilled phrase will be "`reveal staff`", not "`open chest`". Once we have all of the phrases, we now want to map these phrases back into admissible actions that can be performed in the real world.

Remember that A_o refers to the admissible actions for original objects, such as ⟨"`sweep`", "`use`"⟩ for a broom. ChatGPT can then identify the most similar admissible action in A_o that best matches the action performed in the imaginary world. For example, a "`broom`" in the real world is mapped to a "`wand`" – see Table 1 for an example

Table 1: Magical Story Example

Topic: Magical World - Saving a Princess
Imaginary Story (First Iteration): Whisperweaver discovers hidden passage. Uncover ancient chest in hidden passage. Open chest to reveal enchanted staff. Also find Crescent Mirror in chest. Wield enchanted staff for enhanced spellcasting. Use Crescent Mirror for scrying and divination. Harness the power of the enchanted staff and mirror to defeat evil forces and save princess.
Simplified Story: 1. Discovers Whisperweaver 2. Uncover Ancient Chest 3. Reveal Enchanted Staff 4. Find Crescent Mirror 5. Wield Enchanted Staff 6. Use Crescent Mirror 7. Harness Enchanted Staff.
Real-World Translation: 1. Wear clothes 2. Open nightstand 3. Use broom. 4. Open dresser 5. Use broom. 6. Open dresser 7. Use broom.
Imaginary Story (Second Iteration): Whisperweaver discovers hidden passage. Uncover ancient chest in hidden passage. Open chest to reveal enchanted staff. Also find Crescent Mirror in chest. Wield enchanted staff for enhanced spellcasting. Use Crescent Mirror for scrying and divination. Discover recipe for elixir with Crescent Mirror. Brew elixir in the cauldron. Use enchanted staff to activate the elixir. Use transformed abilities from elixir to defeat the evil threat.

of an imaginary story. If the "`wand`" is used to cast a spell, ChatGPT would determine which admissible action would be most similar to the action "`cast a wand`". If "`sweep`" is chosen, then "`cast a wand`" will be mapped into "`sweep broom`" – see Table 1 once again and the mapping from *Simplified Story* to *Real-World Translation*. The agent can then use these mapped admissible actions to interact with the real-world environment.

4 Text Adventure Games

Text Adventure Game is the testbed to show how the agent does imaginary play in the real world. Text games show the event happening within the current scene by depicting the existing objects and happened actions in short sentences. Objects taken as entities in the game contain the states and actions used in presetting the interactions with the agent. The story might ignore details when events happen, but the text game can record the hidden state changes with words. For this reason, we utilize text games as test playgrounds.

We evaluate the performance of an imaginary story by whether the agent can perform all input actions sequentially, which indicates that the model-translated action sequences can function as guidance to the agent in imaginary play. The reward of each round of the game equals the score of the last action. To tell the game result directly, we set the last action in the sequence as the win state. Thus, if the reward equals the score of the preset win state, the agent successfully finishes all actions given in the sequence.

We developed our text game in TextWorld (Côté et al. 2018), an open-source, extensible engine that both generates and simulates text games. In the game, we mimic the physical environment by mapping out the house floor plan and including pre-scripted interactions with each object in the room to give guidance under different use cases (Narasimhan, Kulkarni, and Barzilay 2015).

4.1 Game Design Details

We design a game by inserting a base map that records the location of each room and logic objects related to different rooms (See Fig 2). Each room has its own furniture and appliances, some of which are required, like a light. Each object O_i has its respective action set A_i which can change the states of itself and the game. For instance, the object `Clothes`, O_{clo}, has two states: "washed" and "not washed". The action `wash` in an action set A_{clo} can convert the state of O_{clo} from "not washed" to "washed" when the agent successfully finishes the action — "`wash cloth`". To execute this action, the agent starts in the *parentBedroom* to grab the dirty clothes. When it moves to the *laundry* and finishes washing the clothes, the agent gains 2 points for `wash clothes`.

For the agent to distinguish similar verbs with similar meanings and reactions, for instance, "`wash cloth`" and "`clean cloth`" should be taken as the same thing to do. In that way, we ensure the agent will take the same reaction every time with synonyms.

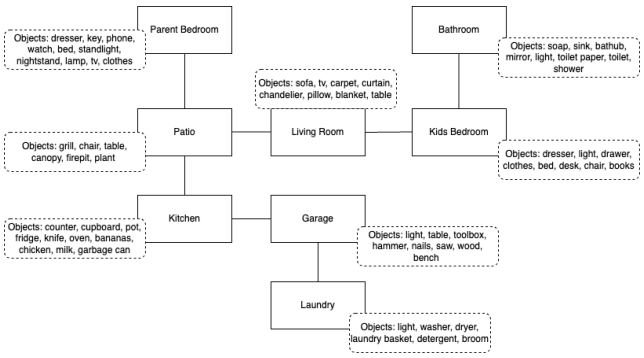

Figure 2: Layout of game "Housework".

4.2 Reinforcement Learning Agent in Text Game

The agent has an action sequence that needs to be finished in the text game and obtains rewards for successfully changing the state of the objects or the game. The game process is as follows: the agent always starts in a fixed room with a given input action sequence. The agent will obtain a reward when it finishes the input action by interacting with the surrounding environment. From section 4.1, if the agent successfully finishes the action "`wash cloth`", it will obtain 2 points as a reward. The reward the agent will gain depends on the difficulty of the action. We categorize the activities into three

levels: stand-alone, interactive, and win, corresponding to 2, 3, and 5 points.

1. We define *stand-alone actions* as actions that the agent is able to finish without using any other objects.For example, when the agent takes the action "`turn on the light`", the `Light` can be turned on directly after locating the `Light`.

2. The definition of *interactive actions* is the actions that the agent is able to interact with other objects. For example, when the agent takes action "`water plant`" and sees the `Plant`, it cannot directly take action if there is no water in its hands. The agent needs to get the kettle, then check whether the kettle is full of water. If not, it will fill the kettle, then carry it to `water plant`.

3. *Win action* is the last action that the agent needs to finish. When the agent takes the last action in the given action sequence, "`clean the oven`", it comes to the kitchen, locates the `Oven`, and then `clean` it. The game ends when the agent successfully goes through the whole action sequence and finishes all actions in it.

5 Findings

Story Generation of Large Language Model:

We used ChatGPT as the LLM to generate stories during our experiment (See Section 3). Most stories required several iterations of revision (Refer to Table 1) until they included the win state in the action sequence. Two limitations of the current model are limited prompting formats and difficulty in understanding interactive actions in the text game.

The first relates to the drawback of the language model is that the generation is uncontrolled. Aside from an initial prompt, generative language models are guided by word co-occurrence, which can lead to repetition, as well as a tendency to focus on descriptive details that do not move a story forward. To solve the problem, we kept crafting prompts to direct the model to create coherent and executable stories with a clear goal and formed a fixed prompting format. The format limits the adaptivity of the agent to varied types of imaginary play. If the setting in imaginary play is modified, the model needs new prompts for the changes.

The other limitation is the difficulty forChatGPT to understand connections between objects in the text game. The generated story cannot associate the objects picked from a previous room with those in the current room if no detailed prompts. That may lead to generating actions not allowed within the text game in the action sequence. The solution to alleviate such problems is to introduce the missing connections into the prompting and have more iterations of story generation to update the prompt with the generated output.

Game Results with Promptings. Results from our sample stories indicate that the agent cannot determine the final win state by itself. To increase the possibility of win, we record the result and feed it back to the language model (ChatGPT). The model knows whether the agent successfully reaches the win state from the previous round's score. If the agent doesn't win, the prompt will tell the model to generate more descriptions of directional information in new stories to guide the agent in the next round. Although new instructions might not give the expected results every time, we still are able to catch the pattern and re-prompting ChatGPT to better train in zero-shot learning.

6 Conclusions

Imaginary play is a creative direction for developing agent learning abilities. With the help of story generation from LLMs (such as ChatGPT (OpenAI 2022)), we can tell the model to generate imaginary play stories that guide the agent's interactions through prompts. Story generation allows the agent to develop interesting imaginative stories with the objects and topic given, allowing the agent to engage in imaginative play in the real world.

We use text games to model what happens within a given story and the interactions the agent generates with the setting, making the interaction controllable and explainable. Through mapping imaginative play to real-world scenarios through text games, we figured out how to use rewards to better prompt the model and construct the stories that can guide the agent in imaginary play.

References

Côté, M.-A.; Kádár, A.; Yuan, X.; Kybartas, B.; Barnes, T.; Fine, E.; Moore, J.; Hausknecht, M.; Asri, L. E.; Adada, M.; et al. 2018. Textworld: A learning environment for text-based games. In *Workshop on Computer Games*, 41–75. Springer.

Martin, L.; Ammanabrolu, P.; Wang, X.; Hancock, W.; Singh, S.; Harrison, B.; and Riedl, M. 2018. Event Representations for Automated Story Generation with Deep Neural Nets. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1).

Narasimhan, K.; Kulkarni, T.; and Barzilay, R. 2015. Language Understanding for Text-based Games Using Deep Reinforcement Learning. arXiv:1506.08941.

OpenAI. 2022. ChatGPT: A Large-Scale Open-Domain Chatbot. https://openai.com/blog/chatgpt/.

Peng, X.; Cui, C.; Zhou, W.; Jia, R.; and Riedl, M. 2023. Story Shaping: Teaching Agents Human-like Behavior with Stories. arXiv:2301.10107.

Peng, X.; Li, S.; Wiegreffe, S.; and Riedl, M. 2022a. Inferring the Reader: Guiding Automated Story Generation with Commonsense Reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, 7008–7029.

Peng, X.; Riedl, M.; and Ammanabrolu, P. 2022. Inherently explainable reinforcement learning in natural language. *Advances in Neural Information Processing Systems*, 35: 16178–16190.

Peng, X.; Xie, K.; Alabdulkarim, A.; Kayam, H.; Dani, S.; and Riedl, M. 2022b. Guiding Neural Story Generation with Reader Models. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, 7087–7111.

Shridhar, M.; Yuan, X.; Côté, M.-A.; Bisk, Y.; Trischler, A.; and Hausknecht, M. 2021. ALFWorld: Aligning Text and Embodied Environments for Interactive Learning. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Wang, R.; Jansen, P.; Côté, M.-A.; and Ammanabrolu, P. 2022. ScienceWorld: Is your Agent Smarter than a 5th Grader?

Zook, A.; Magerko, B.; and Riedl, M. 2011. Formally modeling pretend object play. In *Proceedings of the 8th ACM Conference on Creativity and Cognition*, 147–156.